Session 14: Debugging & Best Practices — Notes & Code (JavaScript)

Mastering Error Types, Syntax/Runtime/Logical Bugs, and Clean Code Practices

October 11, 2026

Contents

1 Topic Notes **2**
- 1.1 Types of Errors . . . 2
- 1.2 Debugging Tips & Best Practices . . . 2

2 Task 1: Fix Buggy Zomato Total Price Code **2**
- 2.1 Buggy Code Analysis . . . 2
- 2.2 Identified Errors . . . 2
- 2.3 Fixed Code . . . 2
- 2.4 Output . . . 3

3 Task 2: Rewrite `isEven` with Indentation & Comments **4**
- 3.1 Original Unreadable Code . . . 4
- 3.2 Improved Code with Documentation . . . 4
- 3.3 Best-Practice Concise Version . . . 4

4 Task 3: `formatFollowersCount(count)` **4**
- 4.1 Code . . . 4
- 4.2 Output . . . 5

5 Task 4: Fix Even Numbers 1–10 Code **6**
- 5.1 Buggy Code . . . 6
- 5.2 Identified Errors . . . 6
- 5.3 Fixed Code . . . 6
- 5.4 Output . . . 6

6 Quick Revision Summary **6**

1 Topic Notes

1.1 Types of Errors

Error Type	When it Occurs	Example
Syntax Error	Code breaks language rules; program won't run	Missing `)`, `;`, or `}`
Runtime Error	Occurs while the program is running	Accessing undefined variable,
Logical Error	Program runs successfully but gives **wrong output**	Using = instead ==, wrong cond

Table 1: Classification of Programming Errors

1.2 Debugging Tips & Best Practices

- **Read Error Messages Carefully:** Stack traces and error logs point directly to the problematic line number.
- **Inspect State:** Use `console.log()` strategically to check variable values at various execution points.
- **Common Pitfalls:** Watch out for assignment (=) vs comparison (===), unclosed brackets, and missing semicolons.
- **Commenting:** Explain *why* code does something (`//` for single-line, `/* */` for multi-line documentation).
- **Indentation & Naming:** Maintain consistent spacing (2 or 4 spaces per block) and use descriptive variable names (e.g., `totalPrice` instead of `t`).

2 Task 1: Fix Buggy Zomato Total Price Code

2.1 Buggy Code Analysis

```
let items = ["Burger", "Pizza", "Fries"];
let prices = [120, 250, 90];
let total = 0;
for (i = 0; i < items.length; i++) {
    total =+ prices[i];
}
console.log("Total price is: " + total);
```

Listing 1: Original Buggy JavaScript Code

2.2 Identified Errors

2.3 Fixed Code

#	Error Snippet	Error Type	Correction
1	`for (i = 0; ...)`	Runtime / Bad Practice	Declare loop variable with `let i = 0`
2	`total =+ prices[i]`	**Logical Error**	Use += instead of =+ (unary plus)

Table 2: Bug Analysis for Zomato Total Price Code

```
// Array of food items ordered from Zomato
let items = ["Burger", "Pizza", "Fries"];

// Corresponding prices for each item
let prices = [120, 250, 90];

// Variable to hold the total price, starts at 0
let total = 0;

// Loop through each item and add its price to total
for (let i = 0; i < items.length; i++) {
    total += prices[i];   // '+=' adds to total (NOT '=+')
}

// Print the final total
console.log("Total price is: " + total);
```

Listing 2: Corrected and Well-Commented Code

2.4 Output

```
Total price is: 460
```

3 Task 2: Rewrite `isEven` with Indentation & Comments

3.1 Original Unreadable Code

```
function isEven(num){if(num%2==0){return true}else{return false}}
```

Listing 3: Unformatted Code

Issues: No indentation, missing semicolons, zero documentation, unnecessarily verbose conditional statements.

3.2 Improved Code with Documentation

```
/**
 * Checks whether a number is even.
 * @param {number} num - The number to check.
 * @returns {boolean} true if even, false otherwise.
 */
function isEven(num) {
    // A number is even when divided by 2 the remainder is 0
    if (num % 2 === 0) {
        return true;
    } else {
        return false;
    }
}

// Testing the function
console.log(isEven(4));   // true
console.log(isEven(7));   // false
```

Listing 4: Cleaned and Commented Function

3.3 Best-Practice Concise Version

```
// Returns true if num is even, otherwise false
function isEven(num) {
    return num % 2 === 0;   // comparison directly evaluates to boolean
}
```

Listing 5: Idiomatic JavaScript Version

4 Task 3: `formatFollowersCount(count)`

4.1 Code

```
/**
 * Formats a follower count the way Instagram displays it.
 * 1500    -> "1.5K"
 * 1200000 -> "1.2M"
 * 900     -> "900"
 * @param {number} count - The follower count.
```

```
 * @returns {string} The formatted count.
 */
function formatFollowersCount(count) {
    // If count is 1 million or more, format as millions (M)
    if (count >= 1000000) {
        return (count / 1000000).toFixed(1) + "M";
    }
    // If count is 1000 or more, format as thousands (K)
    else if (count >= 1000) {
        return (count / 1000).toFixed(1) + "K";
    }
    // Below 1000, return the number as-is
    else {
        return String(count);
    }
}

// Testing the function
console.log(formatFollowersCount(1500));      // "1.5K"
console.log(formatFollowersCount(1200000));   // "1.2M"
console.log(formatFollowersCount(900));       // "900"
console.log(formatFollowersCount(25000));     // "25.0K"
```

Listing 6: Instagram-Style Follower Count Formatter

4.2 Output

```
1.5K
1.2M
900
25.0K
```

Note: `toFixed(1)` rounds the division result to 1 decimal place.

5 Task 4: Fix Even Numbers 1–10 Code

5.1 Buggy Code

```
for (let i = 1; i <= 10; i++) {
    if (i % 2 = 0) {
    console.log(i
```

Listing 7: Buggy Loop Code

5.2 Identified Errors

#	Error Snippet	Error Type	Correction
1	`i % 2 = 0`	Syntax / Logical Error	Use strict comparison `===` instead of `=`
2	`console.log(i`	Syntax Error	Add missing closing parenthesis `)`
3	Missing `}`	Syntax Error	Close block braces for `if` and `for`

Table 3: Bug Analysis for Even Numbers Loop

5.3 Fixed Code

```
// Loop through numbers 1 to 10
for (let i = 1; i <= 10; i++) {
    // Check if the number is even (remainder 0 when divided by 2)
    if (i % 2 === 0) {
        console.log(i);   // print the even number
    }
}
```

Listing 8: Corrected Even Numbers Loop

5.4 Output

```
2
4
6
8
10
```

6 Quick Revision Summary

Core Takeaway: Writing robust code requires proactive error checking, strict comparison operators, clear commenting, and adherence to clean formatting standards.

Concept / Practice	Key Takeaway
Error Categories	Syntax (won't run), Runtime (crashes), Logical (wrong output)
Assignment vs Comparison	= assigns values; === compares them
Variable Declaration	Always use `let` or `const` to avoid global leaks
Arithmetic Assignment	+= accumulates values; avoid mixing up with =+
Code Readability	Consistent indentation and JSDoc comments improve maintainability
Number Formatting	`toFixed(n)` rounds numbers for clean displays (e.g., `1.5K`)

Table 4: Debugging & Best Practices Quick Reference